Лабораторная работа 2 "Работа с базами данных и ORM Sequelize"

1. ЦЕЛЬ РАБОТЫ

Освоение подключения реляционной базы данных PostgreSQL к Node.js-приложению, использование ORM Sequelize для создания моделей и выполнения CRUD-операций, изучение миграций и связей между таблицами **для продолжения разработки серверной части вашего курсового проекта** (начатой в лабораторной работе №1).

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Основы SQL и реляционных баз данных.
- Установку и настройку PostgreSQL.
- Понятие ORM (Object-Relational Mapping).
- Базовые операции Sequelize: define модели, sync, create, findAll, findByPk, update, destroy.
- Миграции и сиды (seeds).

---

3. ЗАДАЧИ РАБОТЫ

- Установить и настроить PostgreSQL (локально или через облачный сервис, например, Neon / Supabase).
- Подключить Sequelize к проекту.
- Создать модель данных, **соответствующую предметной области вашего курсового проекта**.
- **Рекомендуется продолжить предметную область из лабораторной работы №1** (например, если вы начали с товаров – продолжайте с товаров, если с задач – с задач и т.п.) для обеспечения плавного перехода к курсовому проекту.

- Реализовать CRUD-операции с использованием Sequelize вместо временного массива.
- Создать и выполнить миграцию для изменения схемы (добавление нового поля).
- Наполнить базу тестовыми данными (seed).

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Подготовка базы данных

1. Установите PostgreSQL (или создайте бесплатный кластер в Neon / Supabase).
2. Создайте базу данных и пользователя. Запишите строку подключения (URL).

Часть 2. Установка зависимостей

```
npm install sequelize pg pg-hstore
npm install --save-dev sequelize-cli
```

Часть 3. Настройка Sequelize

1. Инициализируйте структуру:
   `npx sequelize-cli init`
2. Настройте `config/config.json` или используйте переменные окружения (`.env`).
   Пример для облачной БД:

   ```
   {
     "development": {
       "use_env_variable": "DATABASE_URL",
       "dialect": "postgres"
     }
   }
   ```
3. Создайте модель и миграцию (например, для модели `Item`):
   `npx sequelize-cli model:generate --name Item --attributes name:string,description:string`

Часть 4. Реализация CRUD в Express

Используйте созданную модель в маршрутах (аналогично лабораторной работе №1), заменив код работы с массивом на вызовы методов Sequelize:

- `GET /items` → `await Item.findAll()`
- `GET /items/:id` → `await Item.findByPk(id)`
- `POST /items` → `await Item.create(req.body)`
- `PUT /items/:id` → `await Item.update(req.body, { where: { id } })`
- `DELETE /items/:id` → `await Item.destroy({ where: { id } })`

Часть 5. Миграция и сиды

1. Создайте миграцию для добавления нового поля (например, `priority` для задач или `stockQuantity` для товаров):
   `npx sequelize-cli migration:generate --name add-field-to-items`
   В файле миграции добавьте колонку с помощью `addColumn`.
2. Выполните миграции: `npx sequelize-cli db:migrate`
3. Создайте seed-файл для начального наполнения:
   `npx sequelize-cli seed:generate --name demo-items`
4. Заполните файл данными (массив объектов) и выполните: `npx sequelize-cli db:seed:all`

---

5. СОДЕРЖАНИЕ ОТЧЕТА

- Титульный лист, цель работы.
- Краткие теоретические сведения об ORM (1–2 абзаца).
- Скриншоты: структура базы данных (pgAdmin, DBeaver или дашборд облачного сервиса), конфигурация Sequelize.
- Листинг модели и контроллеров CRUD (ключевые фрагменты).
- Скриншоты запросов через Postman с ответами из базы данных.
- Демонстрация миграции (показать добавленное поле в таблице).
- Ответы на контрольные вопросы.

- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка `lab12`).

Индивидуальное выполнение. Вариант Темы определяется по **Генератору вариантов.**

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (10 вопросов)

1. Что такое ORM? Какие преимущества и недостатки его использования?
2. Как Sequelize преобразует названия моделей в имена таблиц?
3. Как выполнить поиск с условием (WHERE) в Sequelize? Приведите пример.
4. Что такое миграции и зачем они нужны?
5. Как реализовать связь один-ко-многим между моделями? Приведите пример из вашего проекта.
6. Как выполнить пагинацию (limit, offset) в Sequelize?
7. Как обработать ошибку уникальности (duplicate entry) при создании записи?
8. Чем отличается `findByPk` от `findOne`?
9. Как добавить кастомный метод в модель Sequelize?
10. Какие типы связей поддерживает Sequelize (`belongsTo`, `hasMany`, `belongsToMany`)?

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Эта работа тесно связана с лабораторной работой №1: она заменяет временное хранение данных в массиве на постоянное хранение в базе данных. **Не удаляйте код маршрутов из ЛР №1** – просто замените логику работы с массивом на вызовы методов Sequelize.

- Если вы продолжите предметную область, начатую в ЛР №1 (например, задачи или товары), у вас уже будет готовая основа для реализации полноценного CRUD API, который затем можно будет использовать в курсовом проекте.
- На следующем этапе (лабораторная работа №3) вы добавите систему аутентификации и авторизации.